

第 4 周实验计划：实现两阶段检索

Coarse-to-Fine

2026.4.6 – 2026.4.12

1 本周定位与总体目标

本周进入真正的 budgeted retrieval 设计阶段。与前几周不同，本周的目标不是继续增加一个 baseline，而是建立一个可控制候选规模的两阶段检索框架，并首次系统分析检索质量与计算成本之间的关系。

本周的核心思想是：先使用低成本的 coarse retriever 从全库中召回有限数量的候选页面，再仅在这些候选页面上执行 full late interaction reranking。这样可以避免对整个语料库进行昂贵的 full multi-vector matching，从而将 multi-vector retrieval 转化为一个可预算、可调节、可分析的检索系统。

本周重点关注如下问题：

- 在不同候选预算下，coarse-to-fine 检索能保留多少 full multi-vector 的效果？
- reranking 成本是否随候选规模近似线性增长？
- 哪一个候选规模能够取得最好的质量-成本折中？
- 质量损失主要来自 coarse 阶段漏召回，还是来自 fine 阶段排序能力不足？

2 本周预期成果

本周结束时，应至少获得以下成果：

表 1: 第 4 周预期成果

成果	目标状态
coarse retrieval + fine rerank 管线	已完整跑通
不同 N 设置下的结果	已形成系统对比
coarse-to-fine 与 full MV 的对照结果	已可比较
平均时延与重排成本记录	已完整统计
第一张质量-成本曲线	已绘制初版
最具性价比的 N 候选区间	已有初步结论

本周关键词为：候选预算、精排效率、质量-成本曲线、Pareto 线索。
候选规模设置建议为：

$$N \in \{10, 20, 50, 100\}$$

若语料库规模较小，也可以调整为：

$$N \in \{5, 10, 20, 50\}$$

3 方法设计：Coarse-to-Fine 两阶段检索

3.1 标准流程

本周固定采用如下两阶段检索流程：

1. 使用低成本 coarse retriever 从全库召回 top-N 页面；
2. 对 top-N 候选页面执行 full late interaction reranking；
3. 输出最终 top-k 排序结果；
4. 评测检索效果、时延和重排成本；
5. 与 full multi-vector retrieval 进行对照分析。

形式化地，两阶段检索可以表示为：

$$\mathcal{C}_N(q) = \text{TopN}_{p \in \mathcal{D}} s_c(q, p)$$

其中，

$$q$$

表示 query,

$$p$$

表示页面,

$$\mathcal{D}$$

表示全库页面集合,

$$s_c(q, p)$$

表示 coarse retriever 的相似度分数,

$$\mathcal{C}_N(q)$$

表示 query 对应的 top-N 候选页面集合。

随后, 在候选集合内执行 late interaction reranking:

$$s_f(q, p) = \text{LateInteraction}(Q, P), \quad p \in \mathcal{C}_N(q)$$

最终输出:

$$\text{Rank}(q) = \text{Sort}_{p \in \mathcal{C}_N(q)} s_f(q, p)$$

该设计的关键优势在于: full late interaction 不再作用于整个语料库, 而是仅作用于经过 coarse 阶段筛选后的有限候选集合, 从而显著降低 reranking 成本。

3.2 Coarse Retriever 选择

本周建议优先使用 Week 2 已经跑通的 single-vector visual retrieval 作为 coarse retriever。

选择理由如下:

- 已有稳定实现, 工程风险较低;
- 已有 baseline 结果, 便于横向比较;
- 可自然构成 single-vector coarse retrieval 与 multi-vector fine reranking 的两阶段框架;
- 有助于清晰分析 coarse 阶段召回能力对最终结果的影响。

3.3 成本记录指标

对每个候选规模 N ，需要同步记录如下成本指标：

表 2: 成本记录指标

指标	说明
coarse retrieval time	粗召回阶段耗时
rerank time	late interaction 精排耗时
total latency	单次检索总耗时
per-query latency	平均每条 query 的耗时
top-N candidate size	coarse 阶段输出候选数量
final rerank size	实际进入 reranking 的候选数量
memory / GPU memory	内存或显存占用，若方便则记录

4 六天任务安排

4.1 第 1 天：明确两阶段检索方案并确定 coarse retriever

目标

搭建 coarse-to-fine 检索框架的整体骨架，明确 coarse 阶段、fine 阶段、候选规模和成本记录方式。

主要任务

- 明确两阶段流程：
 - coarse retriever 从全库召回 top-N 页面；
 - full late interaction 仅对 top-N 候选页面精排；
 - 输出最终 top-k；
 - 记录效果和时延。
- 选定 coarse retriever：
 - 推荐使用 Week 2 的 single-vector visual retrieval；
 - 保证与 Week 3 的 full MV reranker 可直接衔接。
- 固定候选规模：

$$N \in \{10, 20, 50, 100\}$$

4. 建立实验日志模板，确保每个 N 都记录效果与成本。

当日产出

- `coarse_to_fine_design.md`
- N 配置表
- coarse retriever 选择说明
- 本周实验日志模板

验收标准

第 1 天结束时，应明确以下内容：

- coarse 阶段使用什么方法；
- fine 阶段如何接入 full MV；
- 本周测试哪些 N；
- 每个 N 需要记录哪些指标。

4.2 第 2 天：实现 coarse retrieval 与 candidate pipeline

目标

完成 coarse retrieval，并为每个 query 输出不同候选深度下的 top-N 候选页面集合。

主要任务

1. 调用 single-vector coarse retriever，从全库召回候选页面。
2. 一次性输出多个候选深度：
 - top-10；
 - top-20；
 - top-50；
 - top-100。
3. 保存统一候选文件 `coarse_candidates.tsv`，字段包括 query 编号、候选页面编号、coarse 排名和 coarse 得分。

表 3: Coarse Candidate 文件字段

字段	说明
query_id	查询编号
candidate_page_id	候选页面编号
coarse_rank	coarse 阶段排名
coarse_score	coarse 阶段得分

Coarse Recall 统计

本日重点不是最终排序，而是检查 coarse 阶段是否足够保 recall。建议统计：

- coarse Recall@10;
- coarse Recall@20;
- coarse Recall@50;
- coarse Recall@100。

若 gold page 在 coarse 阶段没有进入 top-N，那么后续 fine reranking 无法恢复该结果。因此，coarse recall 是两阶段检索系统的上限约束。

当日报出

- coarse_candidates.tsv
- coarse 阶段召回统计表
- 粗召回失败样例记录

验收标准

第 2 天结束时，应确认：

- coarse retriever 是否能稳定召回相关页面；
- 哪个 N 起能够避免明显 recall 损失；
- 哪些 query 在 coarse 阶段已经丢失答案。

4.3 第 3 天：在 top-N 候选上接入 full late interaction 精排

目标

完成 coarse-to-fine 管线的 second-stage reranking, 让 full MV 只在候选集合内执行 late interaction。

主要任务

1. 设计 rerank 输入接口：
 - 输入文件为 `coarse_candidates.tsv`;
 - 每个 query 对应 top-N 页面;
 - query 表示与 Week 3 保持一致;
 - page multi-vector embeddings 复用 Week 3 结果。
2. 对每个 N 独立输出 run 文件：
 - `c2f_N10_run.tsv`
 - `c2f_N20_run.tsv`
 - `c2f_N50_run.tsv`
 - `c2f_N100_run.tsv`
3. 先在小子集上验证：
 - query 是否拿到正确候选;
 - rerank 是否只发生在候选集合内;
 - 最终 top-k 输出是否合理。
4. 检查排序逻辑：
 - coarse 排名较低但相关的页面, 是否能被 fine reranker 提升;
 - 视觉相似但语义不相关的页面, 是否被压低。

当日报出

- 各个 N 对应的 rerank run 文件
- coarse-to-fine 管线脚本
- 子集验证日志

验收标准

第 3 天结束时，应拥有完整的两阶段系统：

coarse retrieval $\rightarrow$ candidate set $\rightarrow$ late interaction rerank $\rightarrow$ top-k output

4.4 第 4 天：系统跑出不同 N 下的效果与时延结果

目标

在正式实验集上运行不同 N 设置，获得效果与时延对比结果。

主要任务

1. 对每个 N 运行完整评测；
2. 记录 Recall@1、Recall@5、Recall@10、MRR@10、nDCG@10；
3. 记录 coarse time、rerank time、total latency、per-query latency；
4. 与 full MV 结果进行对照。

核心结果表

表 4: Full MV 与 Coarse-to-Fine 检索结果对比

Method	Recall@10	MRR@10	nDCG@10	Latency
Full Multi-vector	—	—	—	—
C2F N=10	—	—	—	—
C2F N=20	—	—	—	—
C2F N=50	—	—	—	—
C2F N=100	—	—	—	—

分析重点

第 4 天重点关注趋势，而不是单个绝对数值：

- 小 N 是否已经接近 full MV；
- N 增大后效果是否趋于饱和；
- rerank time 是否随 N 线性或近线性增长；
- coarse-to-fine 的总时延是否显著低于 full MV。

当日产出

- 所有 N 对应的结果文件
- c2f_summary.xlsx
- 初版效果-时延对比表

验收标准

第 4 天结束时，应首次清楚看到：小候选集是否已经能够逼近 full multi-vector retrieval。

4.5 第 5 天：绘制第一张质量-成本曲线并找出最佳 N

目标

将第 4 天的实验结果转化为论文图，绘制 coarse-to-fine 的质量-成本曲线，并判断最具性价比的候选规模。

主要任务

1. 绘制质量-成本图：
 - 横轴：Latency、rerank cost 或 total cost；
 - 纵轴：Recall@10 或 nDCG@10。
2. 标记关键点：
 - full MV 点；
 - single-vector 点；
 - best trade-off 点；
 - 最小延迟点；
 - 接近饱和点。
3. 分析最佳 N：
 - 效果损失是否较小；
 - 时延节省是否明显；
 - rerank 成本是否显著下降。
4. 按 query type 进行补充分析：

- 哪类 query 对小 N 最敏感;
- 哪类 query 在 coarse 阶段最容易丢失证据页;
- 哪类 query 即使 N 很小也能稳定命中。

质量-成本图占位

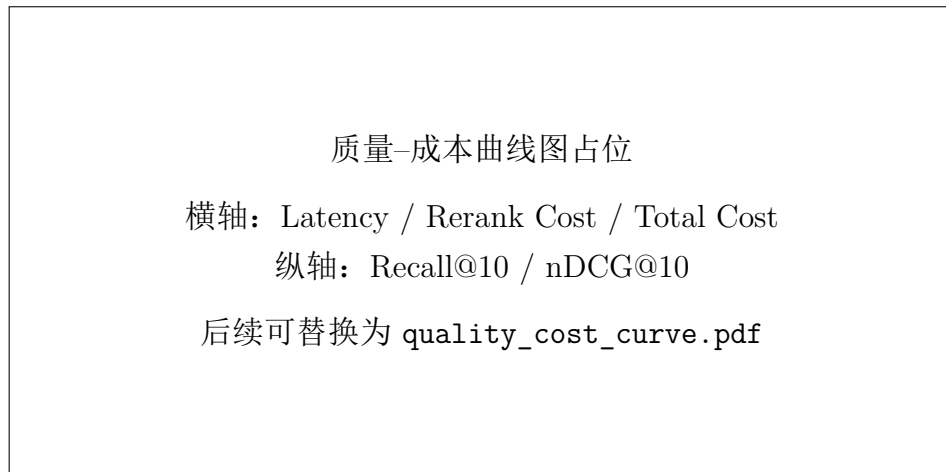


图 1: Coarse-to-Fine 检索的质量-成本曲线。横轴表示检索成本或时延, 纵轴表示 Recall@10 或 nDCG@10。图中应标注 full MV、single-vector 以及不同 N 下的 C2F 结果点。

预期结论模板

可形成如下结论:

当 $N=20$ 或 $N=50$ 时, coarse-to-fine 已能保留 full multi-vector retrieval 的大部分检索效果, 同时显著降低 reranking 成本, 说明适当的候选预算能够带来较优的质量-效率折中。

当日报出

- 第一张质量-成本曲线图
- 最佳 N 分析笔记
- 图注草稿

验收标准

第 5 天结束时, 应能够明确回答:

- 哪个 N 最有性价比;

- 质量损失主要发生在哪个阶段；
- coarse-to-fine 是否值得继续推进。

4.6 第 6 天：整理结果、分析失败案例并写本周实验小结

目标

将本周工作从“方法能跑”整理为“论文能写”，完成结果归纳、失败模式分析与 Week 5 的设计过渡。

典型案例整理

建议整理三类典型案例。

A. C2F 明显逼近 full MV 的案例 这类案例说明：

- coarse 阶段已经成功召回 relevant page；
- fine 阶段将 relevant page 排到前列；
- 候选集较小但最终效果很好。

B. 小 N 失败、大 N 成功的案例 这类案例说明：

- 问题主要不在 rerank；
- 主要失败原因是 coarse 阶段在小 N 下漏召回；
- 增大候选预算后，fine reranker 能恢复正确排序。

C. 即使大 N 仍失败的案例 这类案例说明可能存在更深层问题：

- query 表示不足；
- 页面局部视觉特征不足；
- 单页检索无法解决跨页证据需求；
- 数据标注或评测目标存在歧义。

周报内容

本周周报建议包括：

- coarse-to-fine 方法实现情况；
- 不同 N 的总体结果；
- 时延与重排成本比较；
- 最佳 N 的初步判断；
- 本周发现的失败模式；
- 对 Week 5 patch/token selection 的启示。

Week 5 过渡逻辑

本周 coarse-to-fine 主要解决的是“候选页面数太多”的问题。自然地，Week 5 可以进一步解决“每页 token 太多”的问题。

因此，论文逻辑可以组织为：

Coarse-to-fine retrieval reduces the number of pages that require expensive multi-vector matching, while patch or token selection further reduces the number of visual tokens involved in each page-level late interaction. Together, these two strategies form a budgeted multi-vector retrieval framework.

当日报出

- weekly_report_week4.md
- 质量-成本曲线图初稿
- 典型案例截图
- Week 5 设计笔记

验收标准

第 6 天结束时，应能够写出如下论文级别结论：

通过将 full late interaction 限制在 top-N 候选页面上，两阶段检索能够在显著降低精排成本的同时保留大部分检索质量，证明 coarse-to-fine 是预算化 multi-vector retrieval 的有效组成部分。

5 实验记录模板

5.1 不同 N 的效果记录

表 5: 不同候选规模下的检索效果

Method	Recall@1	Recall@5	Recall@10	MRR@10	nDCG@10
Single-vector	—	—	—	—	—
Full MV	—	—	—	—	—
C2F N=10	—	—	—	—	—
C2F N=20	—	—	—	—	—
C2F N=50	—	—	—	—	—
C2F N=100	—	—	—	—	—

5.2 不同 N 的成本记录

表 6: 不同候选规模下的检索成本

Method	Coarse Time	Rerank Time	Total Latency	Per-query Latency
Single-vector	—	—	—	—
Full MV	—	—	—	—
C2F N=10	—	—	—	—
C2F N=20	—	—	—	—
C2F N=50	—	—	—	—
C2F N=100	—	—	—	—

5.3 Coarse Recall 记录

表 7: Coarse 阶段召回能力

Retriever	Recall@10	Recall@20	Recall@50	Recall@100
Single-vector coarse retriever	—	—	—	—

6 本周核心分析问题

本周结果分析应围绕以下几个问题展开：

1. Coarse-to-fine 在多大程度上接近 full MV？

2. 当 N 增大时，效果提升是否迅速饱和？
3. rerank time 是否与 N 近似线性相关？
4. 最优 N 是由效果决定，还是由时延预算决定？
5. 失败案例主要是 coarse 阶段漏召回，还是 fine 阶段排序失败？
6. 本周结论如何支持 Week 5 的 patch/token selection 设计？

7 本周总结结论

本周的目标是实现并验证一个可控制候选规模的 coarse-to-fine retrieval 框架。该框架首先使用低成本 single-vector retriever 从全库中召回 top-N 候选页面，然后仅在候选集合内执行 full late interaction reranking。通过系统比较不同 N 下的检索效果、时延和 reranking 成本，可以得到第一条质量-成本曲线，并初步判断最具性价比的候选预算。

预期论文级结论如下：

通过将 full late interaction 限制在 top-N 候选页面上，两阶段检索能够在显著降低精排成本的同时保留大部分检索质量。实验结果表明，适当的候选预算可以形成有效的质量-效率折中，证明 coarse-to-fine retrieval 是预算化 multi-vector retrieval 框架中的关键组成部分。

进一步地，本周结果将自然引出下一阶段研究：如果 coarse-to-fine 主要减少需要精排的页面数量，那么 Week 5 的 patch/token selection 将进一步减少每个页面内部参与 late interaction 的 token 数量，从而形成更加完整的 budgeted multi-vector retrieval 方法。